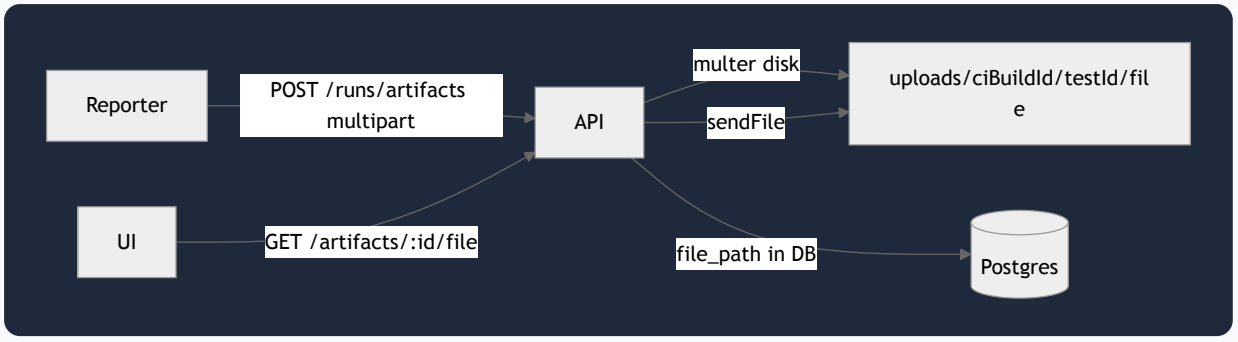
AWS S3 Artifact Storage — Implementation Guide

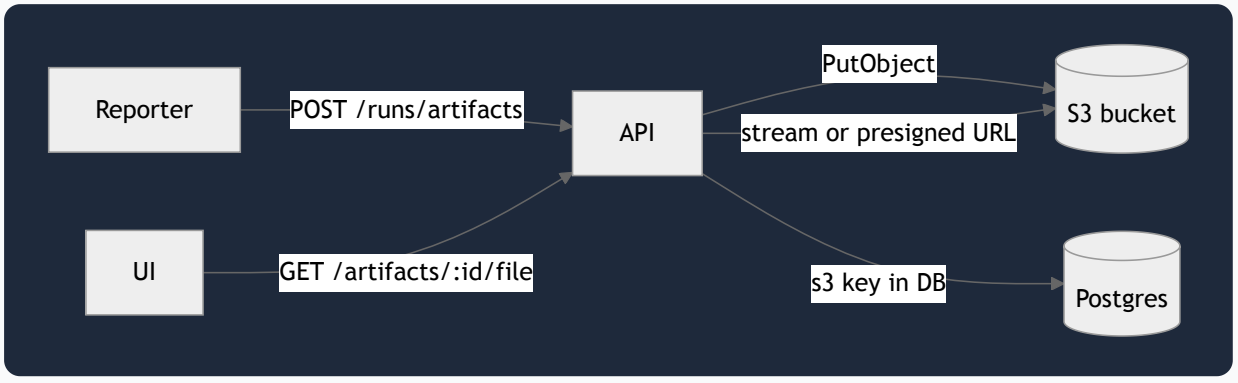
Practical plan for adding **S3 artifact storage** to `playwright-reporter-backend`. Postgres stays as-is; only **screenshots, videos, and traces** move from `uploads/` to S3.

Current flow (what you have)



Reporter package does not need changes — same API, same multipart upload.

Target flow



Step 1 — AWS setup (do this first)

1. **Create S3 bucket**
   Example: `rocketium-playwright-artifacts`
   Region: same as your backend (e.g. `ap-south-1`).
2. **Keep bucket private** — block all public access (default).
3. **Create IAM policy** (minimal):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::rocketium-playwright-artifacts/artifacts/*"
    }
  ]
}
```

4. **Attach to:**
   - **Local dev:** IAM user + access keys
   - **Prod/CI server:** IAM role on EC2/ECS (no keys in env)
5. **Optional lifecycle rule** — delete objects under `artifacts/` after 30–90 days.

Step 2 — Backend env vars

Add to `.env`:

```
ARTIFACT_STORAGE=s3          # local | s3
AWS_REGION=ap-south-1
AWS_S3_BUCKET=rocketium-playwright-artifacts
AWS_S3_PREFIX=artifacts      # optional folder prefix
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
# S3_ARTIFACT_URL_MODE=proxy  # proxy (default) | redirect (presigned URL)
# S3_PRESIGN_EXPIRES_SEC=3600
```

Keep `ARTIFACT_STORAGE=local` until S3 code is done.

Step 3 — Code changes (backend)

3a. Add storage abstraction

New file: `src/storage/artifact-storage.ts`

```
interface ArtifactStorage {
  upload(params: {
    key: string;
    body: Buffer | Readable;
    contentType: string;
  }): Promise<{ key: string; sizeBytes: number }>;

  getReadable(key: string): Promise<{ body: Readable; contentType?: string }>;

  getPresignedUrl?(key: string, expiresSec: number): Promise<string>;
}
```

Implement:

- `LocalArtifactStorage` — current `uploads/` behavior
- `S3ArtifactStorage` — `@aws-sdk/client-s3` + `@aws-sdk/lib-storage` (for large videos up to 200 MB)

3b. S3 object key layout

Mirror your local paths:

```
artifacts/{ciBuildId}/{testId}/{timestamp}-{name}
```

Example:

```
artifacts/26999331755/abc123-testId/1738123456789-screenshot.png
```

Store this key in `artifacts.file_path` (or prefix with `s3:` so old local rows still work).

3c. Change upload route (`routes/runs.ts`)

Today: `multer.diskStorage` → local file → `store.addArtifact(..., file.path)`.

Change to:

1. `multer.memoryStorage()` **or** temp disk + stream
2. Build S3 key from `ciBuildId`, `testId`, filename
3. `artifactStorage.upload(...)`
4. `store.addArtifact(..., s3Key, sizeBytes)`
5. Delete temp file if you used disk

Use `@aws-sdk/lib-storage` `Upload` for streaming — avoids loading 200 MB videos fully into RAM.

3d. Change download route

`GET /api/v1/artifacts/:id/file`:

Mode	Behavior
proxy (simplest)	`GetObject` from S3 → pipe to `res` (same URL as today)
redirect	302 to presigned URL (less load on API, good for UI)

Detect storage from `file_path`:

- starts with `s3:` or matches S3 key pattern → S3
- else → local file (backward compatible)

3e. Schema (optional, later)

For now, **reuse** `file_path` as storage key. Later you can add:

```
ALTER TABLE artifacts ADD COLUMN storage TEXT NOT NULL DEFAULT 'local';
```

Not required for v1.

3f. Wire in `index.ts`

```
const artifactStorage = createArtifactStorage({ uploadsDir, ...s3Config });
createRunsRouter(store, artifactStorage);
```

Remove or keep `express.static('/uploads')` only for legacy local files.

Step 4 — Packages to install

```
cd playwright-reporter-backend
pnpm add @aws-sdk/client-s3 @aws-sdk/lib-storage @aws-sdk/s3-request-presigner
```

Step 5 — Test plan

1. `ARTIFACT_STORAGE=local` — confirm nothing breaks
2. `ARTIFACT_STORAGE=s3` — run one failing test locally (screenshot + video upload)
3. Check S3 console for object under `artifacts/{ciBuildId}/...`
4. Open `GET /api/v1/artifacts/{id}/file` in browser
5. Run `pnpm summary` — artifact names still show; file served from S3

Step 6 — CI / GitHub Actions

No test repo changes. On the **hosted backend**:

- Set `ARTIFACT_STORAGE=s3` + bucket env vars
- Use IAM role (preferred) or GitHub secrets if backend runs in Actions

Reporter still sends files to your API; the API uploads to S3.

Recommended implementation order

Order	Task	Effort
1	AWS bucket + IAM	~30 min
2	`ArtifactStorage` interface + S3 client	~2 h
3	Update POST `/runs/artifacts`	~1 h
4	Update GET `/artifacts/:id/file`	~1 h
5	Env config + docs	~30 min
6	Optional: migrate old `uploads/` to S3	later

Decisions to make before coding

1. **Presigned URL vs proxy through API?**
   - Proxy: simpler, same URLs, API pays bandwidth
   - Presigned: better for future UI at scale
2. **One bucket for all envs or separate?**
   - e.g. `rocketium-playwright-artifacts-staging` / `-prod`
3. **Retention** — auto-delete artifacts after N days via S3 lifecycle?

What you don’t need to change

- Postgres schema (minimal)
- `playwright-rocketium-reporter`
- `automation-tests-2.0` reporter config
- Test upload flow (still multipart to same endpoint)

Next step: Implement Step 3 in `playwright-reporter-backend` — `ArtifactStorage` abstraction, S3 upload/download, and `ARTIFACT_STORAGE=local|s3` toggle with backward compatibility for existing local artifacts.
